

RISC-V RV32I Pipeline CPU — Design Plan

1. Overview

- ISA: RISC-V RV32I subset (~37 instructions, well past the 16 minimum)
- Pipeline: 5-stage in-order (IF → ID → EX → MEM → WB)
- Language: synthesizable Verilog HDL (IEEE 1364, portable between Icarus and Vivado)
- Dev/verify: Icarus Verilog (iverilog) + GTKWave on macOS
- Final: Xilinx Vivado 2019.2 (XSim simulation + synthesis for timing/resource report)
- Reference model: Spike (riscv-isa-sim) for diff-testing; a small Python assembler generates the instruction-memory image (no heavyweight GNU toolchain required)

Design rule (borrowed from Hennessy & Patterson): get the single-cycle machine functionally correct FIRST, then pipeline it. Correct, then fast.

2. ISA subset (RV32I)

Formats: R, I, S, B, U, J.

Group	Instructions	Count
R-type (opcode 0110011)	add, sub, sll, slt, sltu, xor, srl, sra, or, and	10
I arith (0010011)	addi, slti, sltiu, xori, ori, andi, slli, srli, srai	9
Load (0000011)	lb, lh, lw, lbu, lhu	5
Store (0100011)	sb, sh, sw	3
Branch (1100011)	beq, bne, blt, bge, bltu, bgeu	6
U-type (0110111/0010111)	lui, auipc	2
Jump (1101111/1100111)	jal, jalr	2
System (1110011)	ecall, ebreak, csrrw/csrrs/csrrc (+i variants), mret	7

Total ≈ 44 encodings; implement ~37 core + CSRs/mret for the interrupt bonus.

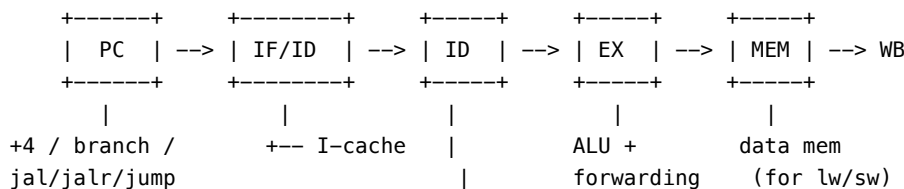
Key encodings

- R-type: funct7[31:25] rs2[24:20] rs1[19:15] funct3[14:12] rd[11:7] opcode
 - sub/sra use funct7=0100000; all others funct7=0000000
- I-type: imm[11:0] rs1 funct3 rd opcode
 - slli/srli/srai: shamt = imm[4:0], upper imm bits must be 0
- S-type: imm[11:5] rs2 rs1 funct3 imm[4:0] opcode (swapped operand order vs. I-type)
- B-type: imm[12|10:5] rs2 rs1 funct3 imm[4:1|11] opcode (imm bit 0 = 0)
- U-type: imm[31:12] rd opcode (lui=0110111, auipc=0010111)
- J-type: imm[20|10:1|11|19:12] rd opcode 1101111 (jal)
- jalr: I-type, opcode 1100111, funct3=000

Register file

32 × 32-bit, x0 hardwired to zero (writes to x0 ignored). x1 = return address for jal/jalr. ABIs: sp=x2, ra=x1, a0-a7=args, t0-t6=saved, s0-s11=saved.

3. Datapath (5-stage)



target mux	regfile	muxes
	read +	
	imm_gen +	
	branch unit	

Stage responsibilities:

- IF: PC select (PC+4 / branch target / jal target / jalr target / mtvec for interrupt), instruction fetch (optionally through I-cache), PC+4 forward.
- ID: register file read (rs1, rs2), immediate generation, branch compare + branch target = PC + B-immediate.
- EX: ALU op (via alu_ctrl), operand muxes (rs1 / forwarded / PC for auipc; rs2 / immediate / forwarded), computes jalr target = (rs1 + I-imm) & ~1.
- MEM: data-memory read/write (byte/half/word with sign/zero extension).
- WB: result select (ALU / load data / PC+4 for jal/jalr / CSR read), register write.

4. Module breakdown

Module	Purpose
pc.v	PC reg + next-PC mux (seq, branch, jal, jalr, mtvec)
if_id.v / id_ex.v / ex_mem.v / mem_wb.v	pipeline registers
regfile.v	32×32, two async read ports, one sync write, x0=0
alu.v	32-bit ALU: add/sub/sll/srl/sra/slt/sltu/xor/or/and + eq/lt/ltu compare out
alu_ctrl.v	funct3 + funct7 + ALUOp → ALU select + branch-compare select
imm_gen.v	6 immediate formats (I/S/B/U/J) → sign-extended 32-bit
control.v	main decode: RegWrite/MemRead/MemWrite/MemtoReg/ALUSrc/Branch/Jump/ALUOp/CSR-op
branch_unit.v	branch taken decision (feeds PC + flush)
forward_unit.v	EX→EX and MEM→EX forwarding (with per-operand mux select)
hazard_unit.v	load-use stall + control/branch flush (stall PC + IF/ID, insert bubble)
csr.v	CSRs + CSR instruction logic (interrupt bonus)
icache.v	direct-mapped I-cache + hit/miss counters (cache bonus)
bht.v	2-bit branch history table + accuracy counter (branch-pred bonus)
cpu_top.v	instantiate + wire all of the above

5. Control signals

Signal	1-wide meaning
RegWrite	write register file at WB
ALUSrc	1 = use immediate (B operand)
MemRead / MemWrite	data-memory read / write
MemtoReg	1 = WB from memory, 0 = from ALU
Branch	instruction is a conditional branch
Jump	jal (WB PC+4, PC=jump target)
JumpReg	jalr
ALUOp[2:0]	drives alu_ctrl (arith/logic/shift/compare/LUI/AUIPC)
WB_PC4	write PC+4 to rd (jal/jalr)
CSRWrite / CSRRead	CSR instruction decode (interrupt)

funct3 and funct7 are passed straight to alu_ctrl. The full per-instruction truth table is generated in control.v (one case per opcode + funct3 group) — see CLAUDE.md for the implementation checklist.

6. Immediate generation (the fiddly RV32I part)

imm_gen.v must produce a sign-extended 32-bit value for every format:

- I: { {20{inst[31]}}, inst[31:20] }
- S: { {20{inst[31]}}, inst[31:25], inst[11:7] }
- B: { {19{inst[31]}}, inst[31], inst[7], inst[30:25], inst[11:8], 1'b0 }
- U: { inst[31:12], 12'b0 }
- J: { {11{inst[31]}}, inst[31], inst[19:12], inst[20], inst[30:21], 1'b0 }

This single module is where most RISC-V beginner bugs live — test it exhaustively before wiring it into the datapath.

7. Hazards

- Data forwarding: EX/MEM result $\rightarrow$ EX (ALU inputs); MEM/WB result $\rightarrow$ EX (load/ALU). Forwarding source priority: MEM $\rightarrow$ EX takes precedence over EX $\rightarrow$ EX (newer data).
- Load-use hazard: a load followed immediately by an instruction consuming it stalls one cycle (hazard_unit asserts stall $\rightarrow$ freeze PC + IF/ID, insert bubble in EX).
- Control hazard: branch/jalr resolved in EX $\rightarrow$ flush IF/ID and ID/EX (2 bubbles for a taken branch without prediction). With the BHT bonus, predict in IF and flush only on mispredict.

8. Bonus items (in difficulty order)

8a. Interrupt (CSRs + mret)

- CSRs: mstatus(0x300), mie(0x304), mtvec(0x305), mscratch(0x340), mepc(0x341), mcause(0x342), mtval(0x343), mip(0x344). Machine mode only.
- Add csrrw/csrrs/csrrc (+ i variants) and mret.
- On external interrupt (testbench-driven signal): save PC $\rightarrow$ mepc, set mcause, jump to mtvec, run ISR, mret restores PC.
- Demo: a timer-like interrupt toggling a counter every N cycles, ISR saves/restores regs.

8b. Branch prediction (2-bit BHT)

- 64-entry 2-bit saturating counter, indexed by PC[7:2].
- Predict in IF; on mispredict flush and update. Expose prediction-accuracy counter.
- Show high accuracy on a loop-heavy program.

8c. Instruction cache (direct-mapped)

- 128 lines $\times$ 16 bytes (2KB), tag = upper PC bits. Hit/miss counters exposed.
- Show hit rate vs. uncached baseline; integrate with a simple memory-mapped main memory.

9. Build order

1. Single-cycle RV32I (whole subset) — pass every instruction test.
2. Insert 4 pipeline registers.
3. Forwarding + load-use stall + branch flush.
4. Diff-test against Spike on real programs (Fibonacci, sort, loop).
5. Interrupt (CSRs + mret).
6. Branch prediction (BHT).
7. I-cache.
8. Performance report + Vivado synthesis.

10. Verification & performance

- Instruction-level: one test per instruction (Python-assembled), checked against expected register/memory state.
- Program-level: Fibonacci, bubble sort, a branch-heavy loop; diff vs. Spike.

- Interrupt: assert external irq, check mepc/mcause and ISR side effects.
- Performance (the “show performance” requirement):
 - $CPI = \text{total cycles} / \text{committed instructions}$ (from XSim/testbench counters).
 - Stall breakdown (load-use, control, structural).
 - Branch-prediction accuracy and cache hit rate (bonus items).
 - Vivado post-synthesis: max frequency (timing), LUT/FF/BRAM utilization.

11. Repo layout (target)

RISCV-CPU/

```

rtl/          # cpu_top.v, pc.v, alu.v, ... all modules
tb/           # testbenches (per-instruction + program-level + irq)
asm/          # test programs (.s) + expected results
tools/asm.py  # Python RISC-V assembler -> instruction memory .hex/.coe
sim/          # Icarus scripts, waveform configs
vivado/       # Vivado project files (created on the Windows/lab machine)
DESIGN.md     # this file
CLAUDE.md     # implementation handoff for Claude Code

```